AI Training

First Semester

Project 1: AI Recruitment Screening Assistant

Project Overview

The goal of this project is to build a small recruitment assistant that helps the HR team screen CVs faster.

The company receives many CVs through its Gmail account whenever a new job opening is available. Instead of HR opening every CV manually, the system should read the job description, understand what the company is looking for, and compare incoming CVs against those requirements.

The system should **help HR make the decision, not make the hiring decision by itself**.

HR should be able to review the candidates, see why a candidate matched or did not match the job requirements, and then decide whether to shortlist, review later, or reject the candidate.

Main Workflow

The system should work roughly like this:

HR creates a job → enters the Job Description → AI extracts the requirements → HR chooses the Must-Have requirements → candidates apply through Gmail → system reads their CVs → candidates are compared with the job → candidates who miss a Must-Have requirement are separated → qualified candidates are ranked → HR reviews the results.

Week 1

Technical Workflow

> **Project Setup → Gmail → Email Processing → CV Files → Text Extraction → Job Description → AI Extraction → Initial Matching**

1. Project Setup

Technologies: Python, VS Code, venv, Git/GitHub, .env

- Create the project folder and open it in VS Code.
- Create a Python virtual environment.
- Install the libraries needed for the project.

- Create a simple project structure.
- Add .env for configuration and .gitignore for files that should not be pushed to GitHub.
- Run a few basic tests to make sure everything is working.

2. Connect Gmail

Technologies: Gmail API, Google Cloud, OAuth 2.0, `google-api-python-client`

Before collecting CVs, the system needs access to the HR Gmail account.

- Understand what an API is and how we will use Gmail API.
- Understand the basic idea behind OAuth.
- Create a Google Cloud project.
- Enable Gmail API.
- Set up OAuth credentials.
- Give the application read-only access to Gmail.
- Write the authentication code.
- Connect the application to Gmail.
- Save the authentication token so the user does not have to log in every time.
- Test the connection and handle basic authentication errors.

3. Understand and Process Gmail Emails

Technologies: JSON, Python Dictionaries, Gmail API, Gmail Search Operators

Once Gmail is connected, the next step is to understand the information returned by the API.

- Look at the structure of a Gmail message.
- Find the sender, subject, message ID, and email body.
- Understand how Gmail represents attachments.
- Search for application emails using Gmail search operators.
- Use filters such as `has:attachment`, `filename:pdf`, and `filename:docx`.
- Decide how the system will recognize emails related to the job.
- Keep track of processed message IDs to avoid processing the same email twice.

4. Find and Download CV Files

Technologies: MIME Types, File Extensions, Base64, Gmail Attachments API

Now the system should be able to find the CV attached to an application email.

- Check whether an email contains an attachment.
- Ignore emails that do not contain files.
- Check the file type.
- Support common CV formats such as PDF and DOCX.
- Ignore unsupported files.
- Download the attachment from Gmail.

- Decode the downloaded file.
- Save it temporarily for the next step.
- Handle emails containing more than one attachment.

Example:

```
Email
  ↓
Has attachment?
  ↓
Yes
  ↓
PDF / DOCX?
  ↓
Yes → Continue
No  → Ignore
```

5. Extract Text from CVs

Technologies: `pypdf`, `python-docx`, Text Processing

The AI cannot work directly with a CV file in the same way it works with plain text, so we first convert the document into text.

- Extract text from PDF files using `pypdf`.
- Extract text from DOCX files using `python-docx`.
- Clean unnecessary spaces and characters.
- Handle empty, corrupted, or unreadable files.
- Keep the extracted text together with basic candidate information.
- Test the extraction using different CVs.

At this point, the system should be able to do:

Gmail → CV File → CV Text

6. Prepare the Job Description

Technologies: Python, Text Files, Basic Data Structures

Before using AI, we need something to compare the CVs against.

- Create a sample Job Description.
- Store it as a text file or simple Python data structure.
- Include information such as:
    - Required skills
    - Experience
    - Education
    - Languages

    - Certifications
- Start thinking about which requirements are essential and which are optional.
- Use the Job Description as the input for the next stage.

7. Use AI to Extract Information

Technologies: Ollama, LLM, Prompt Engineering, Structured Output, JSON

The goal is to turn unstructured text into information that the system can actually work with.

From the Job Description:

- Extract required skills.
- Extract required experience.
- Extract education requirements.
- Extract languages and certifications.
- Organize the results in a consistent structure.

From the CV:

- Extract candidate skills.
- Extract work experience.
- Extract education.
- Extract languages.
- Extract certifications.

Students will:

- Connect Python to Ollama.
- Write prompts for information extraction.
- Ask the LLM to return structured results.
- Test the prompts with different CVs and Job Descriptions.
- Improve the prompts when the output is inconsistent.

8. Initial CV-to-Job Matching

Technologies: LLM, Rule-Based Comparison, Embeddings, Hugging Face, Cosine Similarity

Now that we have:

Job Requirements

+

Candidate Information

we can start comparing them.

- Compare the candidate with each job requirement.
- Decide whether the candidate:

    - **Meets** the requirement
    - **Partially Meets** the requirement
    - **Does Not Meet** the requirement
- Ask the AI to provide evidence from the CV to support the results.
- If the CV does not contain enough information, clearly indicate that **no evidence was found** instead of assuming the candidate has the required skill or experience.
- Introduce **embeddings** to compare similar meanings, not just exact words.
- Use a Hugging Face embedding model such as **sentence-transformers/all-MiniLM-L6-v2**
- Generate embeddings for the relevant job requirement and candidate information.
- Use **Cosine Similarity** to measure how semantically similar the candidate's experience is to the requirement.
- Combine the semantic similarity results with the structured/rule-based comparison.
- Test the matching process with different CVs and requirements, including cases where the wording is different, but the meaning is similar.

9. Automate the CV Screening Process

Technologies: Python, Gmail API, pypdf, python-docx, Ollama

- Connect the previously developed components into one workflow.
- Collect CVs from Gmail and process supported PDF and DOCX files.
- Extract text and candidate information automatically.
- Connect each candidate to the relevant Job Description.
- Avoid processing the same email or CV multiple times.
- Handle invalid files and processing errors without stopping the entire workflow.

10. Finalize Candidate Matching

Technologies: Python, Ollama, Rule-Based Matching, Sentence Transformers

- Compare each candidate's information against the job requirements.
- Evaluate each requirement separately.
- Use rule-based checks for clear conditions and semantic similarity when the wording differs.
- Identify Must-Have requirements that are not satisfied.
- Provide evidence from the CV to explain each matching result.
- Test the system using different CVs and Job Descriptions.

11. Filter and Rank Candidates

Technologies: Python, Pandas, Rule-Based Filtering, Scoring and Ranking

- Separate Must-Have requirements from Preferred requirements.
- Identify candidates who meet all Must-Have requirements.
- Keep candidates who do not meet the mandatory criteria available for HR review.
- Rank qualified candidates according to their remaining requirements.
- Display a configurable shortlist, such as the Top 20 candidates.
- Keep the matching results and ranking available for the user interface.

12. Organize and Save the Results

Technologies: Python, JSON or CSV or use SQLite, File Handling

- Organize candidate information and matching results into a consistent structure.
- Save the results so they can be retrieved by the user interface.
- Keep track of the original CV file for each candidate.
- Avoid creating duplicate candidate records when processing the same application again.
- Test that saved results can be loaded correctly.

13. Build the Backend API

Technologies: FastAPI, Python, Pydantic, Uvicorn

- Create a FastAPI application and organize the existing Python functions into reusable modules.
- Understand how API endpoints allow different parts of an application to communicate.
- Create endpoints to manage jobs, process CVs, and retrieve candidate results.
- Use Pydantic to validate incoming data and structure API responses.
- Connect the API to the existing CV processing and matching functions.
- Handle errors such as invalid job information, missing files, and failed processing requests.
- Run the application using Uvicorn.

14. Build the User Interface

Technologies: Streamlit, Python, Requests, Pandas

- Create a page for entering a Job Description and creating a job.
- Display the requirements extracted by the AI.
- Allow HR to review requirements and select Must-Have criteria.
- Add a button to start CV screening through the API.
- Display application statistics and candidate rankings.
- Allow HR to open candidate details and review the original CV, matching results, missing requirements, and supporting evidence.
- Add options for HR to mark candidates as Shortlist, Review Later, or Reject.
- Use the Python `requests` library to send HTTP requests to FastAPI endpoints and display the returned results.
- Organize the interface into multiple pages using Streamlit.

15. Testing and Final Submission

Technologies: Python, FastAPI TestClient, Git, GitHub, README

- Test the complete workflow from Gmail collection to displaying candidates in the interface.
- Test API endpoints with valid and invalid inputs.
- Test PDF and DOCX extraction, duplicate detection, and processing errors.
- Verify Must-Have filtering, candidate ranking, and evidence generation.
- Fix errors and improve the user experience.
- Write a README explaining the project, installation, configuration, and execution steps.

- Upload the final project to GitHub.
- Ensure credentials, OAuth tokens, and `.env` files are not committed to the repository

****The completed project will be submitted via a GitHub repository.**

Your deadline on Friday 12 PM